

DSA Interview Prep — Concepts, Problems & Python Solutions

A complete companion to your 70-question sheet: for every problem you get the approach, the Python solution, and time/space complexity. Concept questions (easy → hard) are included per topic to test understanding, not just memorization.

1. Arrays

Concept Questions

- **Easy:** Why is indexing $O(1)$ but inserting at the front $O(n)$ for a Python list?
- **Medium:** Explain the two-pointer technique and when it beats a nested loop.
- **Hard:** Why does Kadane's algorithm work — what invariant does `current_sum` maintain?

1.1 Two Sum

Approach: Use a hashmap to store `value -> index` as you scan; check if `target - num` was already seen.

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

Complexity: $O(n)$ time, $O(n)$ space.

1.2 Best Time to Buy and Sell Stock

Approach: Track the minimum price seen so far; at each step compute profit if sold today.

```
def max_profit(prices):
    min_price = float('inf')
    best = 0
    for p in prices:
        min_price = min(min_price, p)
        best = max(best, p - min_price)
    return best
```

Complexity: $O(n)$ time, $O(1)$ space.

1.3 Product of Array Except Self

Approach: Build prefix products, then multiply by suffix products in a second pass — no division used.

```
def product_except_self(nums):
    n = len(nums)
    res = [1] * n
    prefix = 1
    for i in range(n):
        res[i] = prefix
        prefix *= nums[i]
    suffix = 1
    for i in range(n - 1, -1, -1):
        res[i] *= suffix
        suffix *= nums[i]
    return res
```

Complexity: $O(n)$ time, $O(1)$ extra space (output excluded).

1.4 Maximum Subarray (Kadane's Algorithm)

Approach: At each index, decide whether to extend the previous subarray or start fresh.

```
def max_subarray(nums):
    best = cur = nums[0]
    for num in nums[1:]:
        cur = max(num, cur + num)
        best = max(best, cur)
    return best
```

Complexity: $O(n)$ time, $O(1)$ space.

1.5 Merge Intervals

Approach: Sort by start time, then merge overlapping intervals in a single pass.

```
def merge_intervals(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged
```

Complexity: $O(n \log n)$ time (sort dominates), $O(n)$ space.

1.6 Rotate Array

Approach: Reverse the whole array, then reverse the two segments — an $O(1)$ space trick.

```
def rotate(nums, k):
    n = len(nums)
    k %= n
    nums.reverse()
    nums[:k] = reversed(nums[:k])
    nums[k:] = reversed(nums[k:])
```

Complexity: $O(n)$ time, $O(1)$ space.

1.7 Set Matrix Zeroes

Approach: Use the first row/column as markers for which rows/columns to zero, avoiding extra space.

```
def set_zeroes(matrix):
    rows, cols = len(matrix), len(matrix[0])
    first_row_zero = any(matrix[0][c] == 0 for c in range(cols))
    first_col_zero = any(matrix[r][0] == 0 for r in range(rows))

    for r in range(1, rows):
        for c in range(1, cols):
            if matrix[r][c] == 0:
                matrix[r][0] = 0
                matrix[0][c] = 0

    for r in range(1, rows):
        for c in range(1, cols):
            if matrix[r][0] == 0 or matrix[0][c] == 0:
                matrix[r][c] = 0

    if first_row_zero:
        for c in range(cols):
            matrix[0][c] = 0
    if first_col_zero:
        for r in range(rows):
            matrix[r][0] = 0
```

Complexity: $O(m*n)$ time, $O(1)$ extra space.

2. Strings

Concept Questions

- **Easy:** Why does building strings with repeated `+=` cost $O(n^2)$ in the worst case?
- **Medium:** How do you track window boundaries in "Longest Substring Without Repeating Characters"?
- **Hard:** What invariant does the two-hashmap sliding window maintain in "Minimum Window Substring"?

2.1 Valid Anagram

Approach: Compare character frequency counts of both strings.

```
from collections import Counter

def is_anagram(s, t):
    return Counter(s) == Counter(t)
```

Complexity: $O(n)$ time, $O(1)$ space (bounded alphabet).

2.2 Longest Common Prefix

Approach: Compare characters column by column across all strings ("vertical scanning").

```
def longest_common_prefix(strs):
    if not strs:
        return ""
    for i, ch in enumerate(strs[0]):
        for s in strs[1:]:
            if i >= len(s) or s[i] != ch:
                return strs[0][:i]
    return strs[0]
```

Complexity: $O(S)$ where S is total number of characters, $O(1)$ space.

2.3 Valid Palindrome

Approach: Two pointers from both ends, skipping non-alphanumeric characters.

```
def is_palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

Complexity: $O(n)$ time, $O(1)$ space.

2.4 Group Anagrams

Approach: Use the sorted version of each word as a hashmap key.

```

from collections import defaultdict

def group_anagrams(strs):
    groups = defaultdict(list)
    for s in strs:
        key = ''.join(sorted(s))
        groups[key].append(s)
    return list(groups.values())

```

Complexity: $O(n * k \log k)$ time (k = avg word length), $O(n * k)$ space.

2.5 Longest Substring Without Repeating Characters

Approach: Sliding window with a hashmap of last-seen index; shrink window when a repeat is found.

```

def length_of_longest_substring(s):
    last_seen = {}
    left = best = 0
    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1
        last_seen[ch] = right
        best = max(best, right - left + 1)
    return best

```

Complexity: $O(n)$ time, $O(\min(n, \text{alphabet}))$ space.

2.6 String to Integer (atoi)

Approach: Manually parse: skip whitespace, handle sign, consume digits, clamp to 32-bit range.

```

def my_atoi(s):
    s = s.strip()
    if not s:
        return 0
    i = 0
    sign = 1
    if s[0] in '+-':
        sign = -1 if s[0] == '-' else 1
        i += 1
    num = 0
    while i < len(s) and s[i].isdigit():
        num = num * 10 + int(s[i])
        i += 1
    num *= sign
    INT_MIN, INT_MAX = -2**31, 2**31 - 1
    return max(INT_MIN, min(INT_MAX, num))

```

Complexity: $O(n)$ time, $O(1)$ space.

2.7 Minimum Window Substring

Approach: Sliding window with two frequency maps; expand right until valid, then shrink left while still valid.

```
from collections import Counter

def min_window(s, t):
    if not s or not t:
        return ""
    need = Counter(t)
    missing = len(t)
    left = start = end = 0

    for right, ch in enumerate(s, 1):
        if need[ch] > 0:
            missing -= 1
            need[ch] -= 1

        if missing == 0:
            while need[s[left]] < 0:
                need[s[left]] += 1
                left += 1
            if end == 0 or right - left < end - start:
                start, end = left, right
            need[s[left]] += 1
            missing += 1
            left += 1

    return s[start:end]
```

Complexity: $O(n + m)$ time, $O(m)$ space.

3. Linked List

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

Concept Questions

- **Easy:** Why is prepending $O(1)$ for a linked list but $O(n)$ for an array?
- **Medium:** Why does the fast/slow pointer always meet if there's a cycle?
- **Hard:** Why is hashmap + doubly linked list the right combo for LRU Cache?

3.1 Reverse Linked List

Approach: Iteratively flip `next` pointers while walking the list.

```
def reverse_list(head):
    prev = None
    while head:
        nxt = head.next
        head.next = prev
        prev = head
        head = nxt
    return prev
```

Complexity: $O(n)$ time, $O(1)$ space.

3.2 Middle of the Linked List

Approach: Slow pointer moves 1 step, fast pointer moves 2 steps; when fast hits the end, slow is at the middle.

```
def middle_node(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

Complexity: $O(n)$ time, $O(1)$ space.

3.3 Merge Two Sorted Lists

Approach: Use a dummy head, repeatedly attach the smaller current node.

```
def merge_two_lists(l1, l2):
    dummy = tail = ListNode()
    while l1 and l2:
        if l1.val <= l2.val:
            tail.next, l1 = l1, l1.next
        else:
            tail.next, l2 = l2, l2.next
        tail = tail.next
    tail.next = l1 or l2
    return dummy.next
```

Complexity: $O(n + m)$ time, $O(1)$ extra space.

3.4 Linked List Cycle

Approach: Floyd's slow/fast pointer — if they ever meet, there's a cycle.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow is fast:
        return True
    return False
```

Complexity: $O(n)$ time, $O(1)$ space.

3.5 Remove Nth Node From End of List

Approach: Two pointers with a gap of n nodes; when the front pointer reaches the end, the back pointer is right before the node to remove.

```
def remove_nth_from_end(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy
    for _ in range(n):
        fast = fast.next
    while fast.next:
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next
    return dummy.next
```

Complexity: $O(n)$ time, $O(1)$ space.

3.6 Reorder List

Approach: Find the middle, reverse the second half, then merge the two halves alternately.

```
def reorder_list(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    prev, cur = None, slow
    while cur:
        cur.next, prev, cur = prev, cur, cur.next

    first, second = head, prev
    while second.next:
        first.next, first = second, first.next
        second.next, second = first, second.next
```

Complexity: $O(n)$ time, $O(1)$ space.

3.7 LRU Cache

Approach: `OrderedDict` gives $O(1)$ get/put while tracking usage order (move-to-end on access, pop oldest on overflow).

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.cache = OrderedDict()
        self.capacity = capacity

    def get(self, key):
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)
```

Complexity: $O(1)$ time per operation, $O(\text{capacity})$ space.

4. Stack & Queue

Concept Questions

- **Easy:** What's the core LIFO vs FIFO distinction between stacks and queues?
- **Medium:** How does a monotonic stack help with "Daily Temperatures"?
- **Hard:** Why is a monotonic deque $O(n)$ for "Sliding Window Maximum" instead of $O(n*k)$?

4.1 Valid Parentheses

Approach: Push opening brackets; on a closing bracket, check it matches the top of the stack.

```
def is_valid(s):
    pairs = {'(': ')', '[': ']', '{': '}'
    stack = []
    for ch in s:
        if ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
        else:
            stack.append(ch)
    return not stack
```

Complexity: $O(n)$ time, $O(n)$ space.

4.2 Implement Queue using Stacks

Approach: Use two stacks — one for input, one for output; reverse into output only when it's empty.

```
class MyQueue:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []

    def push(self, x):
        self.in_stack.append(x)

    def pop(self):
        self.peak()
        return self.out_stack.pop()

    def peek(self):
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
        return self.out_stack[-1]

    def empty(self):
        return not self.in_stack and not self.out_stack
```

Complexity: $O(1)$ amortized per operation.

4.3 Min Stack

Approach: Push `(value, current_min)` pairs so the minimum is always $O(1)$ to retrieve.

```

class MinStack:
    def __init__(self):
        self.stack = []

    def push(self, val):
        cur_min = min(val, self.stack[-1][1]) if self.stack else val
        self.stack.append((val, cur_min))

    def pop(self):
        self.stack.pop()

    def top(self):
        return self.stack[-1][0]

    def get_min(self):
        return self.stack[-1][1]

```

Complexity: $O(1)$ time per operation, $O(n)$ space.

4.4 Daily Temperatures

Approach: Monotonic decreasing stack of indices; pop when a warmer day is found.

```

def daily_temperatures(temperatures):
    res = [0] * len(temperatures)
    stack = []
    for i, t in enumerate(temperatures):
        while stack and temperatures[stack[-1]] < t:
            j = stack.pop()
            res[j] = i - j
        stack.append(i)
    return res

```

Complexity: $O(n)$ time, $O(n)$ space.

4.5 Next Greater Element I

Approach: Precompute next-greater for every element of `nums2` with a monotonic stack, then look up for `nums1`.

```

def next_greater_element(nums1, nums2):
    next_greater = {}
    stack = []
    for num in nums2:
        while stack and stack[-1] < num:
            next_greater[stack.pop()] = num
        stack.append(num)
    return [next_greater.get(num, -1) for num in nums1]

```

Complexity: $O(n + m)$ time, $O(n)$ space.

4.6 Evaluate Reverse Polish Notation

Approach: Push numbers; on an operator, pop two operands, apply it, push the result.

```
def eval_rpn(tokens):
    stack = []
    ops = {
        '+': lambda a, b: a + b,
        '-': lambda a, b: a - b,
        '*': lambda a, b: a * b,
        '/': lambda a, b: int(a / b),
    }
    for tok in tokens:
        if tok in ops:
            b, a = stack.pop(), stack.pop()
            stack.append(ops[tok](a, b))
        else:
            stack.append(int(tok))
    return stack[0]
```

Complexity: $O(n)$ time, $O(n)$ space.

4.7 Sliding Window Maximum

Approach: Monotonic decreasing deque of indices; the front always holds the current window's max.

```
from collections import deque

def max_sliding_window(nums, k):
    dq = deque()
    res = []
    for i, num in enumerate(nums):
        while dq and nums[dq[-1]] < num:
            dq.pop()
        dq.append(i)
        if dq[0] <= i - k:
            dq.popleft()
        if i >= k - 1:
            res.append(nums[dq[0]])
    return res
```

Complexity: $O(n)$ time, $O(k)$ space.

5. Trees

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

Concept Questions

- **Easy:** What's the difference between a Binary Tree and a BST?
- **Medium:** What does the recursive return value represent when finding the LCA?
- **Hard:** Why can deep recursive trees risk a Python stack overflow, and how would you go iterative?

5.1 Maximum Depth of Binary Tree

Approach: Recursively take 1 + max depth of left/right subtrees.

```
def max_depth(root):
    if not root:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

Complexity: $O(n)$ time, $O(h)$ space (h = height).

5.2 Same Tree

Approach: Recursively compare values and both subtrees.

```
def is_same_tree(p, q):
    if not p and not q:
        return True
    if not p or not q or p.val != q.val:
        return False
    return is_same_tree(p.left, q.left) and is_same_tree(p.right, q.right)
```

Complexity: $O(n)$ time, $O(h)$ space.

5.3 Invert Binary Tree

Approach: Recursively swap left and right children.

```
def invert_tree(root):
    if not root:
        return None
    root.left, root.right = invert_tree(root.right), invert_tree(root.left)
    return root
```

Complexity: $O(n)$ time, $O(h)$ space.

5.4 Binary Tree Level Order Traversal

Approach: BFS with a queue, processing one level at a time.

```

from collections import deque

def level_order(root):
    if not root:
        return []
    res, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        res.append(level)
    return res

```

Complexity: $O(n)$ time, $O(n)$ space.

5.5 Lowest Common Ancestor of a Binary Tree

Approach: Recurse both sides; if a node finds both p and q on different sides, it's the LCA.

```

def lowest_common_ancestor(root, p, q):
    if not root or root == p or root == q:
        return root
    left = lowest_common_ancestor(root.left, p, q)
    right = lowest_common_ancestor(root.right, p, q)
    if left and right:
        return root
    return left or right

```

Complexity: $O(n)$ time, $O(h)$ space.

5.6 Validate Binary Search Tree

Approach: Recurse while tracking valid (low, high) bounds for each node.

```

def is_valid_bst(root, low=float('-inf'), high=float('inf')):
    if not root:
        return True
    if not (low < root.val < high):
        return False
    return (is_valid_bst(root.left, low, root.val) and
            is_valid_bst(root.right, root.val, high))

```

Complexity: $O(n)$ time, $O(h)$ space.

5.7 Serialize and Deserialize Binary Tree

Approach: Pre-order traversal, using a sentinel (e.g. "#") to mark `None` children so it's reversible.

```

class Codec:
    def serialize(self, root):
        vals = []
        def dfs(node):
            if not node:
                vals.append('#')
                return
            vals.append(str(node.val))
            dfs(node.left)
            dfs(node.right)
        dfs(root)
        return ','.join(vals)

    def deserialize(self, data):
        vals = iter(data.split(','))
        def build():
            val = next(vals)
            if val == '#':
                return None
            node = TreeNode(int(val))
            node.left = build()
            node.right = build()
            return node
        return build()

```

Complexity: $O(n)$ time and space.

6. Binary Search

Concept Questions

- **Easy:** What's the prerequisite dataset property for binary search to work?
- **Medium:** How does binary search still work on a rotated sorted array?
- **Hard:** What's the key partition insight in "Median of Two Sorted Arrays"?

6.1 Binary Search

Approach: Classic halving search on a sorted array.

```
def search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Complexity: $O(\log n)$ time, $O(1)$ space.

6.2 Search Insert Position

Approach: Binary search for the first index where `nums[i] >= target`.

```
def search_insert(nums, target):
    left, right = 0, len(nums)
    while left < right:
        mid = (left + right) // 2
        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid
    return left
```

Complexity: $O(\log n)$ time, $O(1)$ space.

6.3 First Bad Version

Approach: Binary search on the answer using the given `is_bad_version` API.

```
def first_bad_version(n, is_bad_version):
    left, right = 1, n
    while left < right:
        mid = (left + right) // 2
        if is_bad_version(mid):
            right = mid
        else:
            left = mid + 1
    return left
```

Complexity: $O(\log n)$ API calls, $O(1)$ space.

6.4 Search in Rotated Sorted Array

Approach: At each step, determine which half is sorted, then check if the target lies within it.

```
def search_rotated(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
    return -1
```

Complexity: $O(\log n)$ time, $O(1)$ space.

6.5 Find Peak Element

Approach: Binary search toward the side with the larger neighbor — a peak is guaranteed to exist there.

```
def find_peak_element(nums):
    left, right = 0, len(nums) - 1
    while left < right:
        mid = (left + right) // 2
        if nums[mid] > nums[mid + 1]:
            right = mid
        else:
            left = mid + 1
    return left
```

Complexity: $O(\log n)$ time, $O(1)$ space.

6.6 Koko Eating Bananas

Approach: Binary search on the eating speed; check feasibility (hours needed) for each candidate speed.

```
import math

def min_eating_speed(piles, h):
    left, right = 1, max(piles)
    while left < right:
        mid = (left + right) // 2
        hours = sum(math.ceil(p / mid) for p in piles)
        if hours <= h:
            right = mid
        else:
            left = mid + 1
    return left
```

Complexity: $O(n \log(\max(\text{piles})))$ time, $O(1)$ space.

6.7 Median of Two Sorted Arrays

Approach: Binary search on the smaller array to find a partition where left-side max $\leq$ right-side min in both arrays.

```

def find_median_sorted_arrays(nums1, nums2):
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1
    m, n = len(nums1), len(nums2)
    left, right = 0, m
    half = (m + n + 1) // 2

    while left <= right:
        i = (left + right) // 2
        j = half - i
        left1 = nums1[i - 1] if i > 0 else float('-inf')
        right1 = nums1[i] if i < m else float('inf')
        left2 = nums2[j - 1] if j > 0 else float('-inf')
        right2 = nums2[j] if j < n else float('inf')

        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 1:
                return max(left1, left2)
            return (max(left1, left2) + min(right1, right2)) / 2
        elif left1 > right2:
            right = i - 1
        else:
            left = i + 1

```

Complexity: $O(\log(\min(m, n)))$ time, $O(1)$ space.

7. Heap / Priority Queue

Concept Questions

- **Easy:** Why does Python's `heapq` only implement a min-heap?
- **Medium:** Why is a min-heap of size k efficient for "Kth Largest Element"?
- **Hard:** What invariant do the two heaps maintain in "Find Median from Data Stream"?

7.1 Kth Largest Element in an Array

Approach: Maintain a min-heap of size k ; the root is the k th largest.

```
import heapq

def find_kth_largest(nums, k):
    heap = nums[:k]
    heapq.heapify(heap)
    for num in nums[k:]:
        if num > heap[0]:
            heapq.heapreplace(heap, num)
    return heap[0]
```

Complexity: $O(n \log k)$ time, $O(k)$ space.

7.2 Top K Frequent Elements

Approach: Count frequencies, then use a heap (or bucket sort) to pick the top k .

```
import heapq
from collections import Counter

def top_k_frequent(nums, k):
    counts = Counter(nums)
    return heapq.nlargest(k, counts.keys(), key=counts.get)
```

Complexity: $O(n \log k)$ time, $O(n)$ space.

7.3 Merge K Sorted Lists

Approach: Push the head of each list into a heap; repeatedly pop the smallest and push its successor.

```
import heapq

def merge_k_lists(lists):
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))

    dummy = tail = ListNode()
    while heap:
        val, i, node = heapq.heappop(heap)
        tail.next = node
        tail = tail.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

Complexity: $O(N \log k)$ time (N total nodes, k lists), $O(k)$ space.

7.4 Find Median from Data Stream

Approach: A max-heap for the lower half and a min-heap for the upper half, kept balanced in size.

```
import heapq

class MedianFinder:
    def __init__(self):
        self.small = [] # max-heap (negated)
        self.large = [] # min-heap

    def add_num(self, num):
        heapq.heappush(self.small, -num)
        heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def find_median(self):
        if len(self.small) > len(self.large):
            return -self.small[0]
        return (-self.small[0] + self.large[0]) / 2
```

Complexity: $O(\log n)$ per insertion, $O(1)$ for median lookup.

7.5 Task Scheduler

Approach: Greedily schedule the most frequent tasks first using a max-heap, respecting the cooldown period.

```
import heapq
from collections import Counter, deque

def least_interval(tasks, n):
    counts = Counter(tasks)
    heap = [-c for c in counts.values()]
    heapq.heapify(heap)
    time = 0
    cooldown = deque() # (available_time, count)

    while heap or cooldown:
        time += 1
        if heap:
            c = heapq.heappop(heap) + 1
            if c < 0:
                cooldown.append((time + n, c))
            if cooldown and cooldown[0][0] == time:
                heapq.heappush(heap, cooldown.popleft()[1])
        if not heap and cooldown:
            time = cooldown[0][0] - 1
    return time
```

Complexity: $O(n \log 26) \sim O(n)$ time, $O(1)$ space (bounded task types).

7.6 Reorganize String

Approach: Greedily place the most frequent character, alternating with the next-most-frequent using a max-heap.

```

import heapq
from collections import Counter

def reorganize_string(s):
    counts = Counter(s)
    heap = [(-c, ch) for ch, c in counts.items()]
    heapq.heapify(heap)
    prev = None
    res = []

    while heap:
        c, ch = heapq.heappop(heap)
        res.append(ch)
        if prev and prev[0] < 0:
            heapq.heappush(heap, prev)
        c += 1
        prev = (c, ch)

    result = ''.join(res)
    return result if len(result) == len(s) else ""

```

Complexity: $O(n \log k)$ time (k = distinct chars), $O(k)$ space.

7.7 IPO

Approach: Sort projects by capital; use a max-heap of profits for all currently affordable projects, greedily picking the most profitable.

```

import heapq

def find_maximized_capital(k, w, profits, capital):
    projects = sorted(zip(capital, profits))
    heap = []
    i = 0
    for _ in range(k):
        while i < len(projects) and projects[i][0] <= w:
            heapq.heappush(heap, -projects[i][1])
            i += 1
        if not heap:
            break
        w += -heapq.heappop(heap)
    return w

```

Complexity: $O(n \log n)$ time, $O(n)$ space.

8. Graphs

Concept Questions

- **Easy:** Adjacency list vs adjacency matrix — space/time tradeoffs?
- **Medium:** Why does a cycle mean no valid topological order (Course Schedule)?
- **Hard:** Why does Dijkstra fail with negative edge weights?

8.1 Number of Islands

Approach: DFS/BFS from every unvisited land cell, sinking the whole connected island.

```
def num_islands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])

    def dfs(r, c):
        if r < 0 or c < 0 or r >= rows or c >= cols or grid[r][c] != '1':
            return
        grid[r][c] = '0'
        dfs(r + 1, c); dfs(r - 1, c); dfs(r, c + 1); dfs(r, c - 1)

    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)
    return count
```

Complexity: $O(mn)$ time, $O(mn)$ space (recursion stack worst case).

8.2 Clone Graph

Approach: DFS/BFS with a hashmap from original node to its clone to avoid infinite loops on cycles.

```
def clone_graph(node):
    if not node:
        return None
    visited = {}

    def dfs(n):
        if n in visited:
            return visited[n]
        copy = type(n)(n.val)
        visited[n] = copy
        for neighbor in n.neighbors:
            copy.neighbors.append(dfs(neighbor))
        return copy

    return dfs(node)
```

Complexity: $O(V + E)$ time and space.

8.3 Flood Fill

Approach: DFS from the start pixel, changing matching-color pixels to the new color.

```
def flood_fill(image, sr, sc, color):
    old_color = image[sr][sc]
    if old_color == color:
        return image
    rows, cols = len(image), len(image[0])

    def dfs(r, c):
        if r < 0 or c < 0 or r >= rows or c >= cols or image[r][c] != old_color:
            return
        image[r][c] = color
        dfs(r + 1, c); dfs(r - 1, c); dfs(r, c + 1); dfs(r, c - 1)

    dfs(sr, sc)
    return image
```

Complexity: $O(mn)$ time, $O(mn)$ space.

8.4 Course Schedule

Approach: Kahn's algorithm (BFS topological sort) — if all courses are processed, no cycle exists.

```
from collections import deque, defaultdict

def can_finish(num_courses, prerequisites):
    graph = defaultdict(list)
    indegree = [0] * num_courses
    for course, prereq in prerequisites:
        graph[prereq].append(course)
        indegree[course] += 1

    queue = deque([c for c in range(num_courses) if indegree[c] == 0])
    visited = 0
    while queue:
        node = queue.popleft()
        visited += 1
        for neighbor in graph[node]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                queue.append(neighbor)
    return visited == num_courses
```

Complexity: $O(V + E)$ time and space.

8.5 Pacific Atlantic Water Flow

Approach: Multi-source BFS/DFS outward from both oceans' border cells; a cell that reaches both is part of the answer.

```

def pacific_atlantic(heights):
    if not heights:
        return []
    rows, cols = len(heights), len(heights[0])
    pacific, atlantic = set(), set()

    def dfs(r, c, visited, prev_height):
        if (r, c) in visited or r < 0 or c < 0 or r >= rows or c >= cols:
            return
        if heights[r][c] < prev_height:
            return
        visited.add((r, c))
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            dfs(r + dr, c + dc, visited, heights[r][c])

    for c in range(cols):
        dfs(0, c, pacific, heights[0][c])
        dfs(rows - 1, c, atlantic, heights[rows - 1][c])
    for r in range(rows):
        dfs(r, 0, pacific, heights[r][0])
        dfs(r, cols - 1, atlantic, heights[r][cols - 1])

    return [list(cell) for cell in pacific & atlantic]

```

Complexity: $O(mn)$ time, $O(mn)$ space.

8.6 Word Ladder

Approach: BFS treating each word as a graph node; an edge exists between words that differ by one letter.

```

from collections import deque

def ladder_length(begin_word, end_word, word_list):
    words = set(word_list)
    if end_word not in words:
        return 0
    queue = deque([(begin_word, 1)])

    while queue:
        word, steps = queue.popleft()
        if word == end_word:
            return steps
        for i in range(len(word)):
            for ch in 'abcdefghijklmnopqrstuvwxyz':
                candidate = word[:i] + ch + word[i + 1:]
                if candidate in words:
                    words.remove(candidate)
                    queue.append((candidate, steps + 1))

    return 0

```

Complexity: $O(N * L^2)$ time (N words, L word length), $O(N * L)$ space.

8.7 Network Delay Time

Approach: Dijkstra's algorithm using a min-heap to always expand the currently-closest unvisited node.

```
import heapq
from collections import defaultdict

def network_delay_time(times, n, k):
    graph = defaultdict(list)
    for u, v, w in times:
        graph[u].append((v, w))

    dist = {}
    heap = [(0, k)]
    while heap:
        d, node = heapq.heappop(heap)
        if node in dist:
            continue
        dist[node] = d
        for neighbor, weight in graph[node]:
            if neighbor not in dist:
                heapq.heappush(heap, (d + weight, neighbor))

    return max(dist.values()) if len(dist) == n else -1
```

Complexity: $O(E \log V)$ time, $O(V + E)$ space.

9. Dynamic Programming

Concept Questions

- **Easy:** What two properties must a problem have to be solvable with DP?
- **Medium:** How do you reduce House Robber's space from $O(n)$ to $O(1)$?
- **Hard:** Why does checking for a subset summing to `total/2` solve Partition Equal Subset Sum?

9.1 Climbing Stairs

Approach: `ways(n) = ways(n-1) + ways(n-2)` — same recurrence as Fibonacci.

```
def climb_stairs(n):
    a, b = 1, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b
```

Complexity: $O(n)$ time, $O(1)$ space.

9.2 House Robber

Approach: At each house, decide `max(rob this house + best two back, skip this house)`.

```
def rob(nums):
    prev, cur = 0, 0
    for num in nums:
        prev, cur = cur, max(cur, prev + num)
    return cur
```

Complexity: $O(n)$ time, $O(1)$ space.

9.3 Coin Change

Approach: Bottom-up DP: `dp[amount]` = min coins to make that amount, built from smaller amounts.

```
def coin_change(coins, amount):
    dp = [0] + [float('inf')] * amount
    for a in range(1, amount + 1):
        for coin in coins:
            if coin <= a:
                dp[a] = min(dp[a], dp[a - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

Complexity: $O(\text{amount} * \text{len}(\text{coins}))$ time, $O(\text{amount})$ space.

9.4 Longest Increasing Subsequence

Approach: Maintain a list of smallest tail values for increasing subsequences of each length; binary search for placement.

```
import bisect

def length_of_lis(nums):
    tails = []
    for num in nums:
        pos = bisect.bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num
    return len(tails)
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

9.5 Partition Equal Subset Sum

Approach: 0/1 knapsack — can we pick a subset summing to exactly half the total?

```
def can_partition(nums):
    total = sum(nums)
    if total % 2:
        return False
    target = total // 2
    dp = [False] * (target + 1)
    dp[0] = True
    for num in nums:
        for t in range(target, num - 1, -1):
            dp[t] = dp[t] or dp[t - num]
    return dp[target]
```

Complexity: $O(n * \text{target})$ time, $O(\text{target})$ space.

9.6 Longest Common Subsequence

Approach: 2D DP where $dp[i][j]$ = LCS length of $\text{text1}[0:i]$ and $\text{text2}[0:j]$.

```
def longest_common_subsequence(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]
```

Complexity: $O(mn)$ time, $O(mn)$ space.

9.7 Edit Distance

Approach: 2D DP where $dp[i][j]$ = min edits to convert $\text{word1}[0:i]$ to $\text{word2}[0:j]$ (insert/delete/replace).

```
def min_distance(word1, word2):
    m, n = len(word1), len(word2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])
    return dp[m][n]
```

Complexity: $O(mn)$ time, $O(mn)$ space.

10. Backtracking

Concept Questions

- **Easy:** What makes backtracking different from plain brute-force recursion?
- **Medium:** How do you avoid duplicate combinations in Combination Sum?
- **Hard:** How would you optimize N-Queens conflict checks to $O(1)$ using sets?

10.1 Subsets

Approach: At each index, branch into "include" and "exclude" this element.

```
def subsets(nums):
    res = []

    def backtrack(start, path):
        res.append(path[:])
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()

    backtrack(0, [])
    return res
```

Complexity: $O(n * 2^n)$ time, $O(n)$ recursion depth.

10.2 Permutations

Approach: At each step, try every unused number, recurse, then undo the choice.

```
def permute(nums):
    res = []

    def backtrack(path, remaining):
        if not remaining:
            res.append(path[:])
            return
        for i in range(len(remaining)):
            path.append(remaining[i])
            backtrack(path, remaining[:i] + remaining[i + 1:])
            path.pop()

    backtrack([], nums)
    return res
```

Complexity: $O(n * n!)$ time, $O(n)$ recursion depth.

10.3 Combination Sum

Approach: Try each candidate (allowing reuse), subtract from target, and prune when $\text{target} < 0$.

```
def combination_sum(candidates, target):
    res = []

    def backtrack(start, path, remaining):
        if remaining == 0:
            res.append(path[:])
            return
        if remaining < 0:
            return
        for i in range(start, len(candidates)):
            path.append(candidates[i])
            backtrack(i, path, remaining - candidates[i])
            path.pop()

    backtrack(0, [], target)
    return res
```

Complexity: $O(n^{\text{target}/\text{min_candidate}})$ time in the worst case.

10.4 Generate Parentheses

Approach: Track open/close counts; only add '(' if open < n, only add ')' if close < open.

```
def generate_parenthesis(n):
    res = []

    def backtrack(path, open_count, close_count):
        if len(path) == 2 * n:
            res.append(''.join(path))
            return
        if open_count < n:
            path.append('(')
            backtrack(path, open_count + 1, close_count)
            path.pop()
        if close_count < open_count:
            path.append(')')
            backtrack(path, open_count, close_count + 1)
            path.pop()

    backtrack([], 0, 0)
    return res
```

Complexity: $O(4^n / \sqrt{n})$ time (Catalan number growth).

10.5 Word Search

Approach: DFS from each cell, marking visited cells and unmarking them on backtrack.

```

def exist(board, word):
    rows, cols = len(board), len(board[0])

    def dfs(r, c, i):
        if i == len(word):
            return True
        if r < 0 or c < 0 or r >= rows or c >= cols or board[r][c] != word[i]:
            return False
        temp, board[r][c] = board[r][c], '#'
        found = (dfs(r + 1, c, i + 1) or dfs(r - 1, c, i + 1) or
                 dfs(r, c + 1, i + 1) or dfs(r, c - 1, i + 1))
        board[r][c] = temp
        return found

    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0):
                return True
    return False

```

Complexity: $O(mn4^L)$ time (L = word length), $O(L)$ space.

10.6 Palindrome Partitioning

Approach: Backtrack over every prefix; only recurse further if the prefix is a palindrome.

```

def partition(s):
    res = []

    def is_palindrome(sub):
        return sub == sub[::-1]

    def backtrack(start, path):
        if start == len(s):
            res.append(path[:])
            return
        for end in range(start + 1, len(s) + 1):
            prefix = s[start:end]
            if is_palindrome(prefix):
                path.append(prefix)
                backtrack(end, path)
                path.pop()

    backtrack(0, [])
    return res

```

Complexity: $O(n * 2^n)$ time in the worst case.

10.7 N-Queens

Approach: Place one queen per row; track occupied columns and diagonals with sets for $O(1)$ conflict checks.

```
def solve_n_queens(n):
    res = []
    cols, diag1, diag2 = set(), set(), set()
    board = [['.'] * n for _ in range(n)]

    def backtrack(row):
        if row == n:
            res.append(''.join(r for r in board))
            return
        for col in range(n):
            if col in cols or (row - col) in diag1 or (row + col) in diag2:
                continue
            cols.add(col); diag1.add(row - col); diag2.add(row + col)
            board[row][col] = 'Q'
            backtrack(row + 1)
            board[row][col] = '.'
            cols.remove(col); diag1.remove(row - col); diag2.remove(row + col)

    backtrack(0)
    return res
```

Complexity: $O(n!)$ time in the worst case, $O(n^2)$ space.

Recommended Study Order

Week	Topics
1	Arrays + Strings
2	Linked List + Stack/Queue
3	Trees + Binary Search
4	Heap + Graphs
5	DP + Backtracking

Tip: For each problem, first explain your approach out loud in under a minute (as you would in an interview), then write the code, then state the time/space complexity — that three-step habit is what interviewers are actually grading.